Computer Networks - Quiz Notes

Scaler Lessons 1-4 | 2-3 hour study pack

Goal: understand the system end-to-end, then memorize the quiz traps, comparisons, formulas, and sequences.

How to use these notes

Pass 1 (about 60-70 min): read the bold ideas and diagrams. Pass 2 (about 60 min): study the tables, sequences, formulas, and quiz traps. Pass 3 (about 20-30 min): answer the rapid-fire questions without looking. Do not spend your limited time memorizing source-code line numbers; memorize what the code is doing.

2h45 Cram Plan

Time	Focus	What to know before moving on
00:00-00:25	Lesson 1: sockets + HTTP	7 syscalls, server/client difference, ports, DNS, byte order, framing
00:25-01:05	Lesson 2: OSI + TCP/UDP/QUIC	encapsulation, headers, handshake/teardown, MSS, SS7, BBR, email/FTP
01:05-01:45	Lesson 3: scaling	fork/thread/event loop, select vs epoll, CGI/FastCGI, limits, Little's Law
01:45-02:25	Lesson 4: nginx	event-loop architecture, sendfile, proxy/cache/routing/upstreams, source-code ideas
02:25-02:45	Quiz pass	answer the self-test from memory; revisit only misses

1. Master Cheat Sheet

Concept	Remember this
TCP server	socket -> bind -> listen -> accept -> read/write -> close
TCP client	socket -> connect -> read/write -> close; kernel chooses ephemeral source port
TCP	reliable, ordered byte stream; application framing is separate
UDP	connectionless datagrams; no delivery/order guarantee
OSI	7 App, 6 Presentation, 5 Session, 4 Transport, 3 Network, 2 Data Link, 1 Physical
Encapsulation	Ethernet contains IP; IP contains TCP; TCP contains application bytes
select	O(watched), rebuild/watch all fds, FD_SETSIZE wall around 1024 in lesson
epoll	kernel-owned interest set; O(ready) event return
CGI	new process per request; env + stdin + stdout/stderr pipes
FastCGI	long-lived process over socket; typed records; empty records delimit streams
Little's Law	concurrency in flight = throughput x latency
nginx	master + worker processes; each worker runs an event loop, usually epoll
sendfile	file -> socket in kernel; avoids user-space copy on the hot path
Reverse proxy	internet/client side -> nginx -> app servers; absorbs slow clients
Cache lock	one request refreshes a popular expired object; others wait
TLS	security layer below the application protocol; symmetric bulk encryption after key setup
QUIC	transport over UDP with independent streams + TLS 1.3; avoids TCP cross-stream HOL
Framing	fixed length, delimiter, length prefix; some protocols use both

One sentence that unifies all 4 lessons

A server is an event-driven consumer of byte streams; the network adds headers around those bytes, protocols define how bytes are framed, and scalable servers avoid paying per-connection setup and per-connection worker costs.

2. Lesson 1 - Network Programming From the Socket Up

Source pages: Lesson 1, pp. 3-48. The deck explicitly starts from a minimal TCP server and ends at custom binary protocol design.

2.1 The seven TCP server system calls

Call	Purpose	Quiz wording
socket()	Create a network endpoint and receive a file descriptor.	Creates the socket; does not assign a listening address by itself.
bind()	Attach a local IP/port to the socket.	Gives the server a known local endpoint.
listen()	Mark the socket as a passive listening endpoint.	Second argument is related to the accept queue.
accept()	Take a completed connection from the listening side and return a new connected fd.	Does not read application data.
read()	Receive bytes from a connected socket.	TCP gives a byte stream, not messages.
write()	Send bytes on the connected socket.	May encounter peer-close/SIGPIPE behavior in the lesson.
close()	Close the descriptor/connection.	Important for cleanup; missing close can contribute to CLOSE_WAIT on the application side.

```
int server_fd = socket(AF_INET, SOCK_STREAM, 0);
bind(server_fd, ...);
listen(server_fd, 1);
while (1) {
   int client_fd = accept(server_fd, NULL, NULL);
   int n = read(client_fd, buf, sizeof(buf));
   write(client_fd, buf, n);
   close(client_fd);
}
```

Important distinction

server_fd is the listening socket. client_fd is the connected socket for one conversation. This distinction is a common quiz/interview trap.

2.2 Ports, privilege, and the accept queue

- **Ports < 1024:** the lesson presents these as privileged/root-only on typical Unix systems.
- **Examples:** 80 HTTP, 443 HTTPS, 25 SMTP, 21 FTP, 110 POP; user-space examples include 3000, 8080, 6379, 2026.
- **Security reason:** binding a low port may require elevated privilege; a vulnerable server running as root increases blast radius.
- **listen(fd, backlog):** backlog is a queue depth, not the total connection capacity.
- **Kernel handshake:** the lesson emphasizes that the TCP handshake can be completed by the kernel and the connection can wait in a queue before accept() returns.
- **Full queue behavior:** the lesson warns that exact full-queue behavior is not portable; some systems may refuse while others may leave the client trying until timeout.
- **Linux backlog cap:** the lesson notes Linux caps the requested backlog at net.core.somaxconn, so the number in code may not equal the effective queue size.

Q: What happens with bind() missing?

A: Nothing owns the intended local endpoint; the lesson describes an immediate connection-refused/RST style failure.

Q: What happens with bind() present but accept() never called?

A: Connections can be established and sit in the kernel queue; the client may appear to hang until timeout.

2.3 Signals and socket shutdown

Item	Meaning
SIGINT (2)	Typical Ctrl-C termination request.
SIGKILL (9)	Cannot be caught/handled.
SIGPIPE (13)	In the lesson, writing to a peer that has already closed/reset can terminate the process by default.
SIGTERM (15)	Polite termination request.
SIGHUP	Often seen when an SSH session disconnects.

SO_LINGER {1,0}

The lesson uses this to demonstrate an abortive close: RST instead of FIN, unsent data can be discarded, and the connection does not follow the normal TIME_WAIT path shown for a polite close. Treat it as a demonstration mechanism, not a default production pattern.

2.4 Client side, ephemeral ports, byte order, DNS

- **Client normally does not bind:** the kernel chooses an ephemeral source port when connect() is used.
- **Endpoint shape:** client 192.168.1.50:48123 -> server 142.x.x.x:443 (illustrative).

- **Ephemeral range in the lesson:** typically 32768-60999 on Linux; finite, and TIME_WAIT can temporarily keep ports unavailable.
- **htons():** host-to-network short; converts a 16-bit value into network byte order.
- **htonl():** host-to-network long/32-bit value.
- **ntohs()/ntohl():** convert network order back to host order.
- **gethostbyname():** hides hostname resolution; the lesson describes /etc/hosts first, then resolver/DNS, caching, and the possibility of blocking. It recommends getaddrinfo() for real code.

```
2026 decimal = 0x07EA
host (little-endian example): EA 07
network byte order:           07 EA
htons(2026) -> network representation
```

Memory hook

IP answers "which machine?" Port answers "which application endpoint on that machine?" The client source port can be temporary; the server usually needs a stable destination port.

2.5 OSI model from the code you already wrote

Layer	Name	Examples from the lesson
7	Application	HTTP, your echo protocol
6	Presentation	encryption, compression, encoding
5	Session	session management, authentication
4	Transport	TCP, UDP, ports
3	Network	IP, routing, ICMP
2	Data Link	Ethernet, Wi-Fi, MAC, ARP, switches
1	Physical	copper, fibre, radio

Common quiz confusion

Ports are Layer 4. IP addresses are Layer 3. MAC addresses/Ethernet are Layer 2. HTTP is Layer 7.

2.6 telnet, netcat, curl, tcpdump, Wireshark

- **telnet:** useful for manually typing text protocols into a TCP connection.
- **nc/netcat:** quick generic TCP/UDP client/listener; ideal for local experiments.
- **openssl s_client:** test encrypted/TLS services such as port 443.
- **curl -i:** shows HTTP response headers; curl -vv exposes much more connection/TLS detail.
- **tcpdump/Wireshark:** capture actual bytes instead of guessing. The lesson uses tcpdump on port 2026 and then ASCII/hex output.

2.7 Many clients: fork vs select vs epoll

Model	Strength	Cost/problem
fork() per connection	Simple; process isolation.	Process memory/scheduler overhead; poor at very high connection counts.
one thread per connection	Cheaper than a process and simpler than an event loop.	Each thread still consumes stack and scheduler state.
select()	Portable multiplexing.	O(n), rebuild/scans descriptors; fixed FD_SETSIZE ceiling in the lesson.
epoll()	Efficient Linux readiness notification.	Linux-specific API; more complex programming model.

select vs epoll

select asks the kernel about a watched set repeatedly. epoll keeps the interest set in the kernel and returns ready events. Remember: select = O(watched); epoll = O(ready).

2.8 Connection pools and socket options

- **Connection pool:** open once, reuse many times, avoiding repeated TCP/TLS setup.
- **Browser connection cap:** the lesson uses an old Chrome example of 6 connections per host; later requests queue when that cap is reached.
- **ulimit -n:** process file-descriptor limit; sockets count toward it.
- **INADDR_ANY:** listen on available local interfaces; convenient in development, a deliberate production choice.
- **SO_REUSEADDR:** useful when restarting servers whose old sockets remain in TIME_WAIT.
- **Promiscuous mode:** lets a capture device observe frames not addressed to it in appropriate network conditions; the lesson uses it to motivate packet sniffing.

2.9 TLS

- **Position:** TLS protects the transport/application exchange; it is not exclusively an HTTP mechanism.
- **Key setup:** asymmetric cryptography/certificates establish trust and shared key material.
- **Bulk data:** symmetric cryptography is used because it is much faster for large amounts of data.
- **Certificate:** binds a public key to an identity through the certificate trust model/CA chain.
- **Pinning:** an application can trust a specific certificate/key rather than relying on the general CA trust store.

2.10 Protocol design and framing

TCP framing rule

TCP does not preserve application message boundaries. The receiver needs a framing strategy.

Framing	How it works	Tradeoff
Fixed length	Every message is exactly N bytes.	Easy parsing, but padding/won't-change-N limitation.
Delimiter	Read until a sentinel/boundary.	Simple and stream-friendly; delimiter needs escaping if it can occur in data.
Length prefix	Read length, then exactly that many bytes.	Binary-safe and robust; sender needs to know/encode size.

2.11 Text vs binary; BCD; ASN.1; TLV; Base64/hex

Topic	Essentials
Text protocol	Human-readable/debuggable; easy with telnet/tcpdump; verbose and parsing can be messy.
Binary protocol	Compact and explicit; needs tooling/hex/base64 for inspection.
BCD	Binary Coded Decimal: 4 bits per decimal digit; exact decimal representation; lesson notes uses in SIM/SMS/card contexts.
ASN.1	Schema/grammar-first; codec generation; used in X.509, LDAP, SNMP, telecom examples in the lesson.
TLV	Type-Length-Value repeated; length makes unknown fields skippable and supports forward compatibility.
Hex	Two hex characters per byte; good for reading/debugging binary.
Base64	3 bytes become 4 printable characters; roughly 33% size overhead; transport encoding rather than a protocol design.
RPC	Marshal arguments + frame + send + demarshal; network failure means it is not a normal local function call.
gRPC	RPC style with protobuf and HTTP/2; binary, multiplexed, schema-first, tooling required.

3. Lesson 2 - OSI, SS7, TCP/UDP/QUIC, and Text Protocols

Source pages: Lesson 2, pp. 1-41. The deck extends the socket view downward into L1/L2 and sideways into SS7, modern transports, and application protocols.

3.1 Encapsulation: one layer's header + body becomes the next layer's payload

```
[Ethernet header][ IP packet ] [FCS]
                 [ IP header ][ TCP segment ]
                              [ TCP header ][ application bytes ]
```

- **Ethernet frame:** destination MAC, source MAC, EtherType, IP payload, FCS.
- **IPv4 packet:** version/IHL, total length, TTL, protocol (6 = TCP in the lesson), checksum, source/destination IP, TCP payload.
- **TCP segment:** source port, destination port, sequence number, ACK number, flags, window, application payload.
- **Application bytes:** HTTP request, TLS record, SMTP command, SIP message, or any other protocol data.

MTU/MSS calculation from the deck

For a 1500-byte Ethernet MTU with a 20-byte IPv4 header and 20-byte TCP header: MSS = 1500 - 20 - 20 = 1460 bytes. The lesson also notes a 1518-byte Ethernet frame after adding the 14-byte Ethernet header and 4-byte FCS.

3.2 Layer 1/2: Wi-Fi behavior

- **CSMA/CA:** Wi-Fi senses the channel, defers if needed, uses random backoff, transmits, and expects an ACK.
- **Why not CSMA/CD?** a radio cannot generally listen cleanly while transmitting because its own signal overwhelms what it is trying to hear.

- **Lesson intuition:** collisions are detected on a wire in the classic Ethernet model; on radio, the protocol instead tries to avoid them.

3.3 OSI layering with long physical paths

- **Satellite example:** the lesson uses a path involving a laptop, local link, satellite terminal, RF uplink, LEO satellite, optional laser crosslink, ground gateway, then the internet.
- **Why it matters:** higher layers do not need to know whether the physical path used copper, radio, fibre, or satellite; the layering boundary hides that implementation detail.

3.4 SS7 - the telecom signalling network

- **Core idea:** move signalling/control out of the voice bearer into a separate packet-switched signalling network.
- **Before SS7:** in-band signalling mixed control with the voice channel.
- **After SS7:** voice uses a bearer channel; signalling uses MTP/SCCP/TCAP/MAP/ISUP.
- **Big systems idea:** this is a control-plane/data-plane separation pattern long before modern software used the same language.

OSI-ish job	SS7 example	TCP/IP example
Application	MAP / INAP / CAP / ISUP	HTTP / DNS / SMTP / SIP
Session-ish	TCAP	TLS/app framing (no distinct OSI layer in the deck)
Transport-ish	SCCP	TCP / UDP
Network	MTP3	IP
Data Link	MTP2	Ethernet / PPP / HDLC
Physical	MTP1 / bearer	Copper / fibre / radio

SS7 vs TCP reliability
The lesson contrasts hop-by-hop reliability in SS7 with end-to-end reliability in TCP. This is a design choice: where in the stack should reliability live?

3.4b SS7 terminology and two flows

Term	Remember
MTP2	Signal units, FSN/BSN, BIB/FIB behavior, CRC-16, retransmission at the signalling-link level.
MTP3	Point-code routing using DPC/OPC; SLS provides load-sharing selection.
SCCP	Subsystem numbers and Global Title Translation; the lesson compares its role with a transport-ish layer.
TCAP	Dialogues/components; ASN.1 BER-style encoding in the stack.
MAP	Mobile application procedures such as roaming and SMS; mo-ForwardSM uses an operation code shown in the deck.
ISUP	Call signalling messages used for circuit call setup/teardown.

- **SMS flow:** handset A -> serving switch/VLR -> SMSC (store-and-forward) -> HLR lookup -> serving MSC B -> handset B; delivery is the SMSC's problem after acceptance/queueing.
- **Call flow:** IAM starts setup, ACM reports address complete/ringback, ANM indicates answer and cuts through the bearer; REL/RLC release the circuit.

3.5 Why SMS is 140 bytes and how it becomes 160 characters

- **140 bytes:** the lesson explains this as a product of stacked SS7/MAP length limits; the smallest relevant limit wins.
- **No segmentation/reassembly in the simple model:** the message is designed to fit in one signalling unit.
- **GSM-7:** 7 bits per character, so 140 bytes x 8 bits / 7 bits per character = 160 characters.
- **UCS-2:** the lesson gives 70 characters in 140 bytes for non-Latin/emoji cases in its simplified calculation.

Quiz arithmetic
140 x 8 = 1120 bits; 1120 / 7 = 160 GSM-7 characters. The byte limit and character limit are not the same number.

3.6 SIP / VoIP

- **Signalling:** SIP is text-shaped and uses messages such as INVITE, 100 Trying, 180 Ringing, 200 OK, ACK, BYE.
- **Media:** RTP/SRTP carries the actual media separately, shown as UDP-based in the deck.
- **Design pattern:** signalling/control path and media/data path are separated.

3.7 TCP reliability mechanics

Mechanism	What to remember
Sequence number	Tracks byte position/order.

ACK number	Acknowledges received sequence space.
Retransmission	Recovers lost data using timers/duplicate ACK behavior as described in the lesson.
Flow control	Receiver window limits how much the sender may have outstanding.
Congestion control	Sender adapts to network capacity; Reno/CUBIC are loss-based examples, BBR uses a different model.
Checksum	Detects corruption at TCP level; the lesson calls out the 16-bit ones-complement checksum.

3.8 TCP 3-way handshake

```
CLIENT                  SERVER
SYN seq=1000  ----------------->
          <----------------  SYN-ACK seq=5000, ack=1001
ACK seq=1001, ack=5001 --------->

ESTABLISHED
```

- **Why 3?** Client announces an initial sequence number; server acknowledges it and announces its own; client acknowledges the server sequence number.
- **SYN/FIN consume sequence numbers:** the lesson explicitly notes this detail.
- **ISNs:** randomized/keyed generation makes blind forged sequence guessing harder.

3.9 SYN floods and SYN cookies

- **Attack:** send many SYNs from spoofed/incomplete sources and fill the finite half-open connection backlog.
- **SYN cookie:** avoid storing full per-connection state before the final ACK; encode enough information in the server sequence number to reconstruct/validate later.
- **Tradeoff:** limited option space means some TCP options may not survive the cookie path; the lesson notes why cookies are commonly activated under pressure.

3.10 TCP teardown, CLOSE_WAIT, TIME_WAIT

```
CLIENT                  SERVER
FIN ---------------------------->
    <--------------------------- ACK
    (server may still send)
    <--------------------------- FIN
ACK ---------------------------->

TIME_WAIT on the active closer before CLOSED
```

- **Half-close is real:** one direction can stop while the other continues sending.
- **CLOSE_WAIT:** the lesson treats persistent CLOSE_WAIT as a strong sign that application code has not closed the socket after the peer closed.
- **TIME_WAIT:** part of orderly TCP teardown; connected to the familiar "address already in use" restart problem and SO_REUSEADDR.

3.11 RTT / connection setup tax

Setup	Lesson intuition
TCP	One round trip before application data in the simplified picture.
TCP + TLS 1.2	More round trips for crypto setup.
TCP + TLS 1.3	Fewer crypto round trips than TLS 1.2.
TCP Fast Open	Can allow data with a SYN for returning clients under supported conditions.
QUIC / HTTP/3	Transport and crypto setup are designed together for a lower handshake cost in the lesson.

HLD intuition

Bandwidth is often purchased by adding capacity; propagation delay is constrained by physics and distance. Every handshake/RTT therefore matters for latency-sensitive paths.

3.12 UDP

- **Header in the deck:** source port, destination port, length, checksum - 8 bytes total.
- **No handshake:** UDP sends datagrams without TCP-style connection establishment.
- **No ordering/reliable retransmission by the transport:** the application can add what it needs.

- **Use cases in the lesson:** DNS, VoIP, video calls, gaming, Zoom/WebRTC-style traffic.

3.13 QUIC and HTTP/3

```
HTTP/3
  |
QUIC  <- independent streams + TLS 1.3
  |
UDP
  |
IP
```

- **Problem with HTTP/2 over TCP:** many streams share one ordered TCP byte stream, so a lost packet can stall delivery of data for all streams - cross-stream head-of-line blocking.
- **QUIC:** each stream has its own sequence space, so loss can be isolated to the affected stream.
- **Userspace:** the lesson emphasizes that QUIC can evolve in application/browser space without waiting for kernel TCP changes everywhere.

3.14 SMTP, MIME, POP3, IMAP, FTP

Protocol	Key quiz points
SMTP	Text-based mail transfer; commands such as EHLO, MAIL FROM, RCPT TO, DATA; a lone dot ends DATA. Lesson ports: 25 relay, 587 submission, 465 implicit TLS.
SMTP envelope vs headers	MAIL FROM/RCPT TO are envelope routing; From:/To: inside DATA are message headers and need not match.
MIME/Base64	Attachments can be carried through text-oriented mail; Base64 adds about 33% size overhead.
POP3	Mailbox download model; USER, PASS, STAT, LIST, RETR, DELE, QUIT. Port 110 in the lesson; POP3S 995 is the TLS-secured variant.
IMAP	Mailbox state remains on server; folders, read/unread flags, search, partial fetch, IDLE-style updates. Port 143 in the lesson.
FTP	Separate control and data connections; passive mode makes both connections client-initiated and works better with NAT/firewalls.

FTP active vs passive

Active: server connects back to client for data. Passive: client opens the extra data connection to the server-selected port. Know why passive is much more firewall/NAT-friendly.

3.15 Framing examples across protocols

Protocol/example	Framing style in the lesson
SMTP	Delimiter: lone dot terminates DATA.
HTTP/1.1 headers	Delimiter: CRLF CRLF ends header block; body length may then be explicit.
MIME	Delimiter: multipart boundary.
SMS / TPDU	Length-oriented.
FastCGI	Length inside records + empty records as stream delimiters.
IMAP literals	Length plus framing around the message.
HTTP/2	Binary framing over a persistent stream.

4. Lesson 3 - One Process, Many Clients

Source pages: Lesson 3, pp. 1-34. The central question is how a single machine keeps many client connections alive while limiting per-connection cost.

4.1 The scaling models

Model	Mental picture	Why it hurts
Fork per request	one process per client	Process image/page tables/fds/scheduler entry per connection.
Thread per request	one thread per client	Thread stack + scheduler entry per connection.
One loop, many sockets	one worker watches many sockets	More complex state machines, but scales much better for idle connections.

What production actually does

The lesson says real systems often stack the ideas: pre-fork/pre-thread a worker pool, then run a select/epoll loop inside each worker. You get process isolation plus event-driven concurrency.

4.2 select() in code

```
FD_ZERO(&readfds);
FD_SET(server_fd, &readfds);
for (clients) FD_SET(client, &readfds);
select(maxfd + 1, &readfds, NULL, NULL, NULL);
if (FD_ISSET(server_fd, &readfds)) accept(...);
for (clients) if (FD_ISSET(client, &readfds)) read/write(...);
```

- **readfds:** sockets with data ready to read / listening socket readiness.
- **writefds:** sockets where writing can proceed.
- **exceptfds:** exceptional conditions in the select interface.
- **timeout:** last argument; NULL means wait indefinitely in the lesson code.
- **O(1) delete trick:** when removing clients from an array, replace the removed entry with the last active client and decrement count.

4.3 The select 1024 wall

- **FD_SETSIZE:** fixed-width bitmap limit highlighted by the lesson as 1024 in the glibc model used by the course.
- **Why ulimit does not save select:** the select bitmap itself is compile-time sized; raising the process fd limit does not expand the bitmap.
- **epoll alternative:** register fds once; epoll_wait returns descriptors that actually fired.
- **Other names mentioned:** kqueue on BSD/macOS, IOCP on Windows, io_uring on modern Linux.

Three names, one wall

The lesson links 1024 to FD_SETSIZE, the common default ulimit, and the K in C10K. Keep the concepts distinct: select has a compile-time representation limit; ulimit is a resource limit; C10K is the scaling problem.

4.4 Timeouts and retries

- **Never wait forever:** servers can be blocked reading/writing/accepting.
- **select timeout:** controls maximum blocking time when non-NULL.
- **Client retry model:** deadline -> timeout -> retry with backoff, instead of immediate infinite retry loops.
- **Why this matters:** one wedged client should not consume an indefinite amount of memory/work/attention.

4.5 CGI

```
HTTP request
  |
 Apache
  |---- envp ----> CGI program
  |---- stdin ---> CGI program
  |<--- stdout --- CGI program
  |<--- stderr --- error log
```

- **CGI lifecycle:** fork/exec a program per request, run it, then discard it.
- **stdin:** request body for POST-style data.
- **environment:** method, query string, server protocol, and transformed request headers.
- **stdout:** CGI response; headers, blank line, then body.
- **stderr:** error/debug output to the server log path.
- **Cost:** repeat process/interpreter startup on every request.

4.6 FastCGI

```
client -> nginx/httpd -> FastCGI socket -> php-fpm (long-lived workers)
```

- **Main improvement:** keep the application process alive; pay setup once.
- **Transport:** Unix/TCP socket between web server and application pool.
- **Request:** typed FastCGI records such as BEGIN_REQUEST, PARAMS, STDIN.
- **Response:** STDOUT, STDERR, END_REQUEST records.
- **Framing:** record header contains a length; an empty PARAMS/STDIN record marks the end of that stream.

Repeated pattern

Prefork, servlets, connection pools, FastCGI: all are examples of amortizing setup - initialize once, reuse many times.

4.7 Servlets

- **Startup:** load classes and instantiate the servlet.
- **Per request:** invoke service(req,res) on a pooled thread.
- **Benefit:** no new process/program load per request.
- **Tradeoff:** isolation is weaker than separate processes; a bad servlet can affect neighbors in the same runtime.

4.8 Web server vs non-web server

HTTP-speaking examples from the lesson	Non-HTTP/own-protocol examples from the lesson
Router admin pages	Redis / RESP
Many Node applications	MySQL wire protocol
Firestore / CouchDB	MQTT brokers
Kafka behind a REST proxy	Kafka native binary protocol

Why this matters

Scaling advice depends on the protocol. HTTP statelessness enables easy request-to-any-server routing; persistent database/protocol connections create a different resource problem.

4.9 HTTP statelessness and horizontal scaling

- **HTTP request:** can contain the information required to answer it, so any healthy server can often handle the next request.
- **Horizontal scaling:** add a server to the pool; stateless requests do not inherently need to return to the same machine.
- **Persistent connections:** Redis/MySQL-like systems prefer reusing connections because connect + handshake + authentication + setup costs money/latency.

4.10 The four scaling walls

Wall	Why it matters
File descriptors	Each socket consumes a descriptor.
Memory	Buffers, stacks, per-connection structures accumulate.
Context switches/syscalls	User/kernel transitions have overhead; repeated small operations add up.
Queueing/latency	Slow work means more requests remain in flight and consume resources.

4.11 C10K, C10M, and per-connection memory

- **C10K:** the 1999 problem of supporting ~10,000 simultaneous connections on one machine.
- **C10M:** the lesson cites a 2013 argument about scaling to ~10 million connections by reducing kernel overhead/bypassing traditional paths.
- **Modern demonstrations:** the lesson cites 2-3 million connections on one server as an order-of-magnitude example.
- **Thread memory:** lesson example: ~8 MB default stack per OS thread; 10,000 threads implies ~80 GB of address-space reservation in that simplified calculation.
- **Goroutine:** lesson example: ~2 KB initial stack, growing on demand; the runtime multiplexes goroutines onto a small number of OS threads using a netpoller.

4.12 Syscall cost, kernel bypass, sendfile, io_uring

- **Syscall tax:** repeated user/kernel transitions add overhead; the lesson presents this as a real consideration at very high operation rates.
- **Kernel bypass examples:** DPDK, netmap, mTCP, Seastar are named as approaches that trade convenience/tools for more control and speed on suitable networks.
- **sendfile():** move file data to a socket through the kernel without copying the data through your user-space buffer.
- **io_uring:** batch/submit operations through shared rings to reduce per-operation syscall overhead.

4.13 Little's Law

Formula to memorize

L = lambda x W. In practical server terms: concurrency in flight = throughput (requests/sec) x latency (sec/request).

Traffic	Latency	In-flight work
100 req/s	0.1 s	10
100 req/s	1.0 s	100
1000 req/s	0.2 s	200

Interpretation
Same traffic + higher latency = more concurrent work to hold. The lesson summarizes this as "slowness is a memory bug."

4.14 Apache, HAProxy, Varnish, nginx - what the lesson wants you to see

Software	Architecture/idea highlighted in the deck
Apache	prefork -> worker -> event. Event parks idle keep-alives on epoll and gives worker threads only actual work.
HAProxy	Started before epoll existed; event-driven select/epoll proxy/load balancer; persistent upstream connections can absorb connection churn.
Varnish	Caching reverse proxy; threads for active work plus epoll/kqueue waiter for idle connections. Lesson example pools default to a minimum of 100 and maximum of 5000 workers.
nginx	master + workers; one event loop per worker; epoll + sendfile; reverse proxy, cache, load balancing, TLS termination, rate limiting. The lesson also notes thread-pool offload for blocking disk reads.

5. Lesson 4 - nginx Deep Dive

Source pages: Lesson 4, pp. 1-36. The lesson connects history, event-loop architecture, caching/routing/upstreams, and a source-code walkthrough.

5.1 History as a constraint story

Era	What the deck emphasizes
1990	CERN httpd on a NeXT - first web server in the lesson history.
1993	NCSA Mosaic = browser; NCSA HTTPd = server; do not confuse them.
1995	Apache continues the NCSA HTTPd patch lineage.
1996	Apache passes NCSA HTTPd and becomes dominant in the lesson timeline.
1997	Servlet model: long-lived runtime + per-request service call.
1999	C10K named; J2EE popularizes multi-layer application-server architecture.
2001	HAProxy arrives.
2002	nginx development begins specifically around the C10K constraint.
2004	First public nginx release; the lesson uses this as its event-loop-era milestone.
2006	Varnish as caching reverse proxy.
2011	nginx 1.0.0 in the lesson history.

Historical exam trap
Mosaic was a browser, not the first web server, and Apache did not descend from Mosaic. The lesson explicitly calls out this common lineage confusion.

5.2 The architectural question nginx changed

- **Apache question:** How can the per-connection worker become cheaper?
- **nginx question:** Why does a connection need a worker at all while it is idle?
- **nginx model:** each worker handles many connections through an event loop; a connection is mostly state until it has work.
- **Goal:** serve many mostly-idle connections without a dedicated thread/process per connection.

5.3 nginx master/worker architecture

```
master
 |
 +-- worker 0 -> epoll/event loop
 +-- worker 1 -> epoll/event loop
 +-- worker N -> epoll/event loop
 +-- cache manager
 +-- cache loader
```

- **Master:** binds/listens, reads config, forks workers, handles signals; does not normally handle application connections.
- **Workers:** handle connections and HTTP processing through event loops.
- **Cache manager/loader:** separate roles shown in the lesson for cache eviction/index warming.

5.4 The nginx event loop in six lines

```
timer = ngx_event_find_timer();
ngx_process_events(cycle, timer, flags);   /* epoll_wait() */
```

```
ngx_event_process_posted(cycle, &ngx_posted_accept_events);
ngx_shmtx_unlock(&ngx_accept_mutex);
ngx_event_expire_timers();
ngx_event_process_posted(cycle, &ngx_posted_events);
```

- **Sleep most of the time:** the worker spends most of its life asleep in the event wait.
- **Timer is an argument:** the worker does not have to poll repeatedly just to discover timeout expiration.
- **Posted events:** handlers can be queued and drained after polling rather than all running inside the poll call.

5.5 Worker accept balancing

- **ngx_accept_disabled:** the lesson shows an arithmetic rule based on connection slots/free connections so a worker that is nearly full can stop competing for new accepts.
- **No central coordinator:** each worker can make the decision from locally available counts.
- **accept_mutex:** the lesson notes it is off by default in the referenced nginx version behavior, with modern Linux facilities such as EPOLLEXCLUSIVE/SO_REUSEPORT helping distribute accepts.
- **Thundering herd:** many workers waking for the same new connection was a historical kernel/userspace problem; the lesson presents newer kernel support as reducing the need for the userspace workaround.

5.6 The stale-event bug

- **epoll_wait returns a batch:** event 7 can still be in the batch after event 1 closes/recycles its fd.
- **FD reuse:** the same integer fd can later refer to a different connection.
- **Solution in the lesson:** nginx carries a generation/instance signal and validates it before using the event.
- **Pointer low-bit trick:** aligned pointers leave a low bit available; the lesson shows nginx borrowing it as a generation flag.

Quiz trap
An fd number is not a stable identity for the lifetime of a process. The integer can be reused. Event-driven code needs protection against stale events.

5.7 sendfile

Approach	Path
read + write	disk -> page cache -> user buffer -> socket buffer -> NIC
sendfile	disk/page cache -> socket buffer -> NIC; data does not enter your user-space buffer

- **Why it helps:** fewer copies through user space and fewer syscalls for static file delivery.
- **Why not always:** if you must inspect/transform bytes (for example, compression or certain TLS/filtering paths), you need access to data in a processing path.
- **Benchmark nuance:** the lesson explicitly warns that a single loopback benchmark may not show wall-clock wins; reduced syscall/copy work is still valuable as concurrency headroom.

5.8 Reverse proxy

```
client / internet
    |
    v
[ nginx ]  <- TLS termination, buffering, routing, caching, LB
    |
    +----> app 1
    +----> app 2
    +----> app 3
```

- **Forward proxy:** sits next to clients and represents them toward the internet.
- **Reverse proxy:** sits next to servers and represents the server side toward clients.
- **nginx benefits:** route by path/host, distribute requests, terminate TLS, cache, fallback, and absorb slow clients.
- **Slow-client idea:** an application server should not have to keep expensive per-connection state for a phone on a very slow link if the proxy can absorb it.

5.9 nginx caching

```
proxy_cache_path temp/cache levels=1:2 keys_zone=demo:10m max_size=100m inactive=60s use_temp_path=off;
```

Directive/feature	Meaning in the lesson
keys_zone	Shared memory for index metadata/structures (not the full cached body).
levels=1:2	Two directory levels for on-disk objects.
max_size	Disk size ceiling for the cache.
inactive	Object can be evicted when not read for the given period, regardless of TTL. Lesson example: 60s.

keys density	Lesson rule-of-thumb: roughly 8,000 cache keys per MB of shared-memory index in its example.
LRU	Least-recently-used queue; eviction takes from the tail when old enough and not in use.
count == 0	The lesson notes this protects an object currently being streamed from being evicted out from under the client.

5.10 Cache stampede and stale content

- **proxy_cache_lock on;** when a hot item expires, one request refreshes while other requests wait rather than all hitting origin at once.
- **proxy_cache_use_stale ...;** serve an older cached response under configured error/update conditions when origin is unhealthy.
- **Why:** availability can be improved by reducing origin thundering-herd pressure and using a stale-but-valid-enough response rather than failing every request.

5.11 nginx location matching

Rule	Behavior
location = /exact	Exact match wins immediately.
location ^~ /images/	Longest prefix; suppresses regex search once selected.
location ~ \.php$	Regex; first matching regex in file order, not longest regex.
location /	Normal/longest prefix used when no regex overrides it.

Why the data structure matters

The lesson explains that prefix locations are represented like a tree, while regex locations are checked from an ordered array. Therefore tree search produces a longest-prefix result; regex scanning naturally obeys first-match-in-order.

5.12 Weighted round robin

- **Weights:** e.g., 3:1 means long-run request distribution should be roughly 3 to 1.
- **Smooth weighted round robin:** the lesson shows current_weight accumulation and choosing the peer with the largest current weight, then subtracting the total weight.
- **Why smooth:** it keeps the same ratio while avoiding large bursts such as A A A B every cycle.

5.13 try_files and fallbacks

```
location / {
    try_files $uri $uri/ /index.html?$args;
}
```

- **Order:** test file, then directory, then fallback target.
- **Fallback:** the last entry is not another candidate to test; it is what happens after prior checks fail.
- **Named location:** can be reached internally from directives such as try_files/error_page, not directly from a URL.
- **SPA use:** common pattern for client-side routes where unknown paths should reach the front-end app.

5.14 nginx source tree

Directory/file	What the lesson says to look for
src/core/	nginx standard-library-like structures: strings, arrays, lists, hashes, rbtree, queue, pool, buffers, config, cycle, connections.
src/event/	event abstraction and platform-specific wait modules such as epoll/select/kqueue.
src/http/	HTTP state machine, phases, upstream framework, directives/modules.
src/os/unix/	OS/syscall integration, sendfile, processes, shared memory, atomics.
src/stream/	raw TCP/UDP/L4 architecture using the same core ideas.
src/mail/	IMAP/POP3/SMTP proxying built on the same event-driven core.
ngx_queue.h	intrusive doubly-linked list; no separate allocation just to link a node.
ngx_palloc.h	request/pool-oriented memory allocation; destroy the whole pool instead of many individual frees.

5.15 Resumable HTTP parser

- **Why state machine?** TCP segments can split an HTTP request line between arbitrary bytes.
- **Resume:** parser state is stored so the next invocation continues exactly where parsing stopped.
- **Lesson example:** many explicit states for method, spaces, URI, schema, host/port, HTTP version and so on.
- **No simple string function:** a non-blocking parser cannot assume the whole request line is present in one buffer or one call.

5.16 nginx request phases

POST_READ -> SERVER_REWRITE -> FIND_CONFIG -> REWRITE -> POST_REWRITE
-> PREACCESS -> ACCESS -> POST_ACCESS -> PRECONTENT -> CONTENT -> LOG

- **Phase engine:** request passes through a sequence of module hooks/checkers.
- **Suspension:** returning from a phase can mean "not done; resume later" when another event fires.
- **Examples from the lesson:** limit_req in PREACCESS; auth_basic in ACCESS; try_files in PRECONTENT; proxy_pass/fastcgi_pass/static in CONTENT.

5.17 FastCGI inside nginx

8-byte FastCGI record header
version | type | requestId | contentLength | padLen | reserved
then: contentData[contentLength] + padding

- **BEGIN_REQUEST:** starts the request.
- **PARAMS:** carries request/environment-like fields; empty PARAMS ends the parameter stream.
- **STDIN:** carries body bytes; empty STDIN ends the body stream.
- **Response:** STDOUT, STDERR, END_REQUEST.

5.18 The seven-server progression in the deck

Stage	Idea
01-fork	fork() per client - understand the cost.
02-thread	pthread per client - cheaper than process, still per-connection.
03-select	one thread, select loop - runs into the 1024 representation wall.
04-epoll	one thread, epoll loop - thousands of idle connections become manageable.
05-sendfile	static-file copy reduction: read/write vs mmap vs sendfile.
06-fastcgi	long-lived application process + binary record protocol.
07-nginx	routing, cache, weights, fallbacks, all integrated into the same event architecture.

6. Quiz Traps and High-Yield Distinctions

Do not confuse	Correct distinction
Listening socket vs connected socket	server_fd accepts; client_fd talks to one client.
Backlog vs total connections	backlog is a queue depth; it is not the lifetime/total connection limit.
TCP vs application messages	TCP is a byte stream; your protocol defines message framing.
IP vs port vs MAC	IP = Layer 3 address; port = Layer 4 endpoint; MAC = Layer 2 local-link address.
select vs epoll	select walks/rebuilds the watched set; epoll keeps interest in kernel and returns ready events.
ulimit vs FD_SETSIZE	ulimit is a resource limit; FD_SETSIZE is select's fixed bitmap width in the lesson model.
CGI vs FastCGI	CGI creates a process per request; FastCGI keeps long-lived worker processes and talks over a socket.
POP3 vs IMAP	POP leans toward download/local state; IMAP keeps mailbox state on server.
SMTP envelope vs From/To headers	MAIL FROM/RCPT TO route the SMTP transaction; From/To are message headers.
FTP active vs passive	Active: server opens data connection to client. Passive: client opens it to the server.
CLOSE_WAIT vs TIME_WAIT	CLOSE_WAIT points toward application cleanup after peer close; TIME_WAIT is an intentional TCP state after active close.
Forward vs reverse proxy	Forward proxy is client-side; reverse proxy is server-side.
Longest prefix vs first regex	nginx prefixes are matched structurally; regex is first match in config order.
TTL vs inactive cache setting	TTL/freshness and inactive/read-idleness are different concepts in the cache lesson.
Weight vs burst pattern	Weighted distribution is long-run ratio; smooth weighted RR also controls how evenly work is spread.
HTTP/2 vs QUIC	HTTP/2 can multiplex over one TCP stream, but TCP loss can stall multiple streams; QUIC gives independent streams.
TLS vs HTTP	TLS is a security layer that can protect many application protocols, not just HTTP.

7. Memorize These Sequences and Numbers

Memory item	Exact thing to recall from the lessons
Server calls	socket -> bind -> listen -> accept -> read -> write -> close
Client essentials	socket -> connect -> write/read -> close
OSI	7 App, 6 Presentation, 5 Session, 4 Transport, 3 Network, 2 Data Link, 1 Physical
TCP open	SYN -> SYN-ACK -> ACK
TCP close	FIN/ACK -> ACK -> FIN/ACK -> ACK; active closer passes through TIME_WAIT
HTTP/1.1 request	request line + headers + CRLF CRLF + body as applicable
SMTP conversation	EHLO -> MAIL FROM -> RCPT TO -> DATA -> ... -> . -> QUIT
POP3 core	USER -> PASS -> STAT/LIST -> RETR -> DELE -> QUIT
FTP	control connection + separate data connection
Framing	fixed length / delimiter / length prefix
Little's Law	L = lambda W
MSS example	1500 - 20 - 20 = 1460 bytes
GSM-7 example	140 x 8 / 7 = 160 chars
Base64	3 bytes -> 4 chars -> about 33% overhead
select	O(watched); FD_SETSIZE ~1024 in the lesson model
epoll	O(ready) event return in the lesson comparison

8. 30-Question Rapid Review

Q: Why does a server call bind()?

A: To associate the socket with a known local address/port.

Q: What does listen() do?

A: Marks the socket as a passive listening endpoint and sets queue-related behavior.

Q: What does accept() return?

A: A new connected socket for one client.

Q: Why does a client normally not bind?

A: The kernel selects an ephemeral source port.

Q: What does htons stand for?

A: Host-to-network short.

Q: What is TCP fundamentally?

A: An ordered, reliable byte stream.

Q: How does an app find message boundaries on TCP?

A: It defines framing: fixed, delimiter, or length-based (or combinations).

Q: What is a port?

A: A Layer 4 endpoint identifier for an application on a host.

Q: What is DNS doing behind gethostbyname/getaddrinfo?

A: Mapping names to addresses using local hosts/resolver/DNS mechanisms.

Q: What are the OSI layers?

A: Application, Presentation, Session, Transport, Network, Data Link, Physical.

Q: Why does Wi-Fi use CSMA/CA?

A: A radio cannot reliably detect collisions while transmitting, so it avoids them and uses ACKs.

Q: What is SS7?

A: A telecom signalling stack/network separated from the voice bearer path.

Q: Why can SMS be 160 GSM-7 characters in 140 bytes?

A: GSM-7 packs 7-bit characters into the 140-byte field.

Q: What does BBR measure conceptually?

A: Path bottleneck bandwidth and round-trip propagation characteristics rather than simply waiting for loss.

Q: What are the TCP handshake messages?

A: SYN, SYN-ACK, ACK.

Q: What is a SYN flood?

A: Filling the server's finite half-open connection state/backlog with incomplete handshakes.

Q: What are SYN cookies?

A: A way to avoid storing full connection state before the final ACK by encoding state in the server sequence number.

Q: Why can CLOSE_WAIT indicate a code bug?

A: The application has not closed the socket after the peer closed.

Q: Why does UDP work for real-time media?

A: Late data can be less useful than missing data; the app can choose its own recovery policy.

Q: What does QUIC change?

A: Independent streams and transport/crypto designed in userspace over UDP.

Q: Why does fork per connection scale poorly?

A: Per-connection process memory, scheduler state, and creation costs multiply.

Q: Why is select limited in the lesson?

A: FD_SETSIZE bitmap representation is around 1024 in the described glibc model.

Q: What is epoll_wait doing?

A: Sleeping until watched fds have ready events and returning the active events.

Q: What is CGI?

A: A per-request external process interface using environment/stdin/stdout/stderr.

Q: What is FastCGI?

A: A long-lived application process pool reached through a socket using typed records.

Q: What is Little's Law?

A: Concurrency in flight = throughput x latency.

Q: Why did nginx matter?

A: It focused on removing the need for a worker per connection and used event-driven workers.

Q: What is sendfile?

A: A kernel-side file-to-socket path that avoids a user-space copy for static data.

Q: What is nginx location precedence?

A: Exact, then ^~ prefix behavior, then regex first-match rules, then normal prefix fallback in the lesson table.

Q: Why is nginx called a reverse proxy?

A: It sits in front of the server side, routing/caching/terminating/absorbing client connections on behalf of upstreams.

9. Final "Did I Miss Anything?" Checklist

- Lesson 1: seven syscalls, echo server, backlog, privileged ports, SIGPIPE, SO_LINGER, persistent reads, ephemeral ports, htons/htonl/ntohs/ntohl, DNS, OSI, telnet/nc/openssl/curl, fork, select, epoll, connection pools, ulimit, INADDR_ANY, SO_REUSEADDR, tcpdump/Wireshark, TLS, text vs binary, framing, BCD, ASN.1, TLV, hex/base64, RPC, gRPC/protobuf.
- Lesson 2: sockets revisited, OSI L1-L7, Wi-Fi CSMA/CA, satellite layering, SS7 architecture and trust model, packet encapsulation, Ethernet/IP/TCP fields, MTU/MSS, SMS 140-byte reasoning, GSM-7 160 chars, SIP/RTP separation, SS7 vs TCP reliability, BBR, TCP open/close, SYN cookies, RTT tax, UDP, QUIC/HTTP3, SMTP, MIME/Base64, POP3, IMAP, framing, FTP active/passive, hands-on protocol tools.
- Lesson 3: fork/thread/event-loop models, select code, FD_SETSIZE, epoll, production worker stacking, timeouts/retries, CGI pipes, environment variables, CGI framing, FastCGI, FastCGI records, servlets, web vs non-web servers, HTTP statelessness, persistent connections, four resource walls, C10K/C10M, thread/goroutine memory, syscall cost, kernel bypass, sendfile/io_uring, Little's Law, Apache/HAProxy/Varnish/nginx comparison.
- Lesson 4: web-server history, Apache evolution, nginx design question, master/worker model, event loop, accept balancing, stale events, select/epoll comparison, sendfile benchmark nuance, forward/reverse proxy, cache index/disk split, LRU, cache lock, stale cache, location precedence, prefix tree vs regex array, smooth weighted RR, try_files fallback, source tree, ngx_queue, ngx_palloc, HTTP parser state machine, request phases, FastCGI record parsing, seven-server progression, homework themes.

Last 5 minutes before the quiz

Write from memory: (1) the 7 server calls, (2) OSI layers, (3) SYN/SYN-ACK/ACK, (4) TCP close + TIME_WAIT, (5) select vs epoll, (6) CGI vs FastCGI, (7) Little's Law, (8) nginx master/worker/epoll, (9) three framing strategies, and (10) exact vs ^~ vs regex vs normal prefix location behavior.